

Received 4 July 2024, accepted 25 July 2024, date of publication 5 August 2024, date of current version 16 August 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3438996

RESEARCH ARTICLE

UniHENN: Designing Faster and More Versatile Homomorphic Encryption-Based CNNs Without `im2col`

HYUNMIN CHOI^{1,2}, JIHUN KIM³, SEUNGHO KIM¹, SEONHYE PARK¹,
JEONGYONG PARK¹, WONBIN CHOI², AND HYOUNGSHICK KIM¹

¹Department of Electrical and Computer Engineering, Sungkyunkwan University, Seoul 03063, Republic of Korea

²NAVER Cloud Security Dev., Seongnam-si 463-824, Republic of Korea

³Department of Mathematics, Sungkyunkwan University, Seoul 03063, Republic of Korea

Corresponding author: Hyoungshick Kim (hyoung@skku.edu)

This work was supported in part by NAVER Cloud Corporation; and in part by Korean Government through Institute of Information & Communications Technology Planning & Evaluation (IITP) under Grant RS-2022-II220688, Grant 2022-0-01199, Grant RS-2023-00229400, Grant RS-2024-00419073, and Grant RS-2024-00436936.

ABSTRACT Homomorphic encryption (HE) enables privacy-preserving deep learning by allowing computations on encrypted data without decryption. However, deploying convolutional neural networks (CNNs) with HE is challenging due to the need to convert input data into a two-dimensional matrix for convolution using the `im2col` technique, which rearranges the input for efficient computation. This restricts the types of CNN models that can be used since the encrypted data structure must be compatible with the specific model. UniHENN is a novel HE-based CNN architecture that eliminates the need for `im2col`, enhancing its versatility and compatibility with a broader range of CNN models. UniHENN flattens input data to one dimension without using `im2col`. The kernel performs convolutions by traversing the image, using incremental rotations and structured multiplication on the flattened input, with results spaced by the stride interval. Experimental results show that UniHENN significantly outperforms the state-of-the-art 2D CNN inference architecture named PyCrCNN in terms of inference time. For example, on the LeNet-1 model, UniHENN achieves an average inference time of 30.089 seconds, about 26.6 times faster than PyCrCNN's 800.591 seconds. Furthermore, UniHENN outperforms TenSEAL, an `im2col`-optimized CNN model, in concurrent image processing. For ten samples, UniHENN (16.247 seconds) was about 3.9 times faster than TenSEAL (63.706 seconds), owing to its support for batch processing of up to 10 samples. We demonstrate UniHENN's adaptability to various CNN architectures, including a 1D CNN and six 2D CNNs, highlighting its flexibility and efficiency for privacy-preserving cloud-based CNN services.

INDEX TERMS Homomorphic encryption, privacy-preserving machine learning, data privacy.

I. INTRODUCTION

The widespread adoption of deep learning has expanded beyond traditional tasks like image classification [48] to diverse applications such as attack detection [45] and resource extraction analysis [47], with machine learning (ML) now playing a crucial role in sensitive sectors including finance, healthcare, and autonomous systems. However, this proliferation has raised significant privacy concerns [34], [51], particularly for cloud-based ML services processing sensitive data on remote servers [4]. Balancing ML's powerful

capabilities with the need to safeguard individual privacy has become a critical challenge, highlighting the urgent need for advanced privacy-preserving techniques in ML applications handling sensitive information.

Homomorphic encryption (HE) is a powerful tool for preserving privacy in ML applications [12], [20], [23], [24], [25], [32]. For example, Choi et al. introduced privacy-preserving biometric authentication systems [53], [54] that leverage HE for web and cloud environments. HE allows computations to be performed on encrypted data without decryption, ensuring that sensitive information remains confidential even when processed by a third party. This is particularly beneficial for cloud-based ML services, where

The associate editor coordinating the review of this manuscript and approving it for publication was Sedat Akleyek^{1b}.

sensitive data is often transmitted to remote servers for processing. By using HE, cloud providers can ensure data privacy, such as a bank training an ML model to detect fraudulent transactions without accessing customer data or a healthcare provider analyzing patient medical records without disclosing patient identities. Additionally, faster inference time with privacy preservation is crucial for real-time applications in these sensitive fields, where quick and secure data processing can significantly improve decision-making and user experience.

Convolution operations are essential to Convolutional Neural Networks (CNNs) but require significant computational resources when performed on ciphertexts in HE. Each kernel shift requires element-wise multiplications and additions, resource-intensive operations on ciphertexts. To reduce the computational load, most HE-based CNN implementations like TenSEAL [7] use the `im2col` function. This function transforms the input data into a matrix without discarding or altering the original information, thus allowing for more efficient computation. TenSEAL is applied in various research areas, including healthcare [22], federated learning [46], and biometrics [49]. However, this approach limits the versatility of CNN models to configurations that accommodate only a single convolution layer, potentially hindering their use in more complex structures. In practical scenarios, encrypted user data could be useful across multiple ML models. For instance, an encrypted patient image stored in a hospital database could be utilized for future analysis by various ML algorithms. Therefore, encrypting data without restricting it to a specific model would offer greater flexibility and utility.

We introduce UniHENN, a privacy-preserving CNN model using HE that enables efficient CNN inference without relying on the `im2col` function. Our novel convolution algorithm flattens input data into a one-dimensional form, eliminating the need for data rearrangement. The kernel traverses the input, performing convolutions with incremental rotations and structured multiplication, ensuring only relevant elements are used. This approach overcomes `im2col` limitations by reducing image size dependency and focusing on kernel size, improving efficiency and flexibility across various ML models. UniHENN also supports batch operations for efficient multi-ciphertext processing. Our key contributions include:

- We introduce UniHENN, a novel CNN model inference mechanism based on HE, which facilitates input ciphertext reusability. Unlike other CNN implementations that require a specific model input structure for the `im2col` function, UniHENN is designed to handle model-free input ciphertexts, enabling the use of encrypted input data across various HE-based ML services without the need for re-encryption. We demonstrate the effectiveness of this approach by successfully constructing and training seven different CNN models on four datasets: MNIST [14], CIFAR-10 [27], USPS [19], and electrocar-

diogram (ECG) [37]. The source code for UniHENN is available at <https://github.com/hm-choi/uni-henn>.

- We empirically demonstrate the efficiency of UniHENN. Experimental results indicate an average inference time of 30.089 seconds, significantly outperforming the state-of-the-art HE-based 2D CNN inference architecture PyCr-CNN [15], which requires an average of 800.591 seconds for inference.
- We introduce a batch processing strategy for UniHENN to handle multiple data instances in a single operation. This strategy efficiently combines multiple data instances into a single ciphertext, reducing the inference time for 10 MNIST images to 16.247 seconds. It outperforms TenSEAL's CNN model [7], which takes 63.706 seconds. While UniHENN is less efficient than TenSEAL for processing a single image, it surpasses TenSEAL's performance when processing multiple images simultaneously, particularly when $k \geq 3$.

II. BACKGROUND

A. CONVOLUTIONAL NEURAL NETWORK (CNN)

Convolutional Neural Networks (CNNs) were first introduced by LeCun et al. [29] for processing grid-structured data like images. In 1989, the LeNet-1 model was presented [28], comprising two convolutional layers, two average pooling layers, and one fully connected layer. Which is the first concept of the LeNet architecture.

In 1998, the LeNet-5 model was presented [30], comprising three convolutional layers, two pooling layers, and two fully connected layers. It was designed to efficiently recognize handwritten postal codes, outperforming traditional methods like the multilayer-perceptron model.

CNNs gained significant traction in the 2010s, primarily due to their excellent performance on large datasets like ImageNet [13], which includes millions of images across 1,000 classes.

A typical CNN consists of an input layer, multiple hidden layers, and an output layer. The input layer receives the image and forwards its pixel values into the network. Since images are generally matrices, the node count in the input layer corresponds to the image size. The hidden layers, situated between the input and output layers, extract relevant features using convolution, activation, and pooling operations. Each hidden layer receives information from either the input or preceding layers, facilitating iterative learning. This information is then passed to subsequent layers, culminating in the final prediction or classification at the output layer.

Kiranyaz et al. [26] introduced a 1D CNN for disease-specific ECG classification. A 1D CNN is a variant of CNNs tailored for one-dimensional data, making it particularly suitable for signal data, time-series data, and text data. In this architecture, both the input and the convolution filter are one-dimensional; the filter slides over the input in the convolution layer to execute operations.

B. HOMOMORPHIC ENCRYPTION (HE)

Homomorphic encryption (HE) is a cryptographic technique that allows computations on encrypted data without the need for decryption. Formally, given messages m_1 and m_2 , an encryption function Enc , and computationally feasible functions f and f' for ciphertext and plaintext, respectively, HE satisfies $f(Enc(m_1), Enc(m_2)) = Enc(f'(m_1, m_2))$.

Several HE schemes, such as BGV [36], GSW-like schemes [8], [10], and CKKS [9], exist, each with different computational needs and data types. In UniHENN, we use the CKKS scheme, which is advantageous for encrypting vectors of real or complex numbers.

CKKS encryption requires structuring plaintext to mirror input data. The number of slots, defined during parameter selection, determines the encryptable vector size. CKKS supports three fundamental ciphertext operations: addition, multiplication, and rotation.

Let N be the total degree of the CKKS parameter, then the number of slots is $N/2$. Two real vectors $\mathbf{v}_1, \mathbf{v}_2$ and $C(\mathbf{v}_1), C(\mathbf{v}_2)$ denote the ciphertext of the vectors $\mathbf{v}_1, \mathbf{v}_2$. (P) indicates that the operation is defined between a ciphertext and a plaintext, and (C) indicates that the operation is defined between two ciphertexts. Then the operations, addition (*Add*), multiplication (*Mul*), and rotation (*Rot*) can be represented as follows:

- Addition (P): $Add(C(\mathbf{v}_1), \mathbf{v}_2) = C(\mathbf{v}_1 + \mathbf{v}_2)$
- Addition (C): $Add(C(\mathbf{v}_1), C(\mathbf{v}_2)) = C(\mathbf{v}_1 + \mathbf{v}_2)$
- Multiplication (P): $Mul(C(\mathbf{v}_1), \mathbf{v}_2) = C(\mathbf{v}_1 \times \mathbf{v}_2)$
- Multiplication (C): $Mul(C(\mathbf{v}_1), C(\mathbf{v}_2)) = C(\mathbf{v}_1 \times \mathbf{v}_2)$
- Rotation: $Rot(C(\mathbf{v}), r) = C(v_r, v_{r+1}, \dots, v_{N/2-1}, v_0, \dots, v_{r-1})$, where $\mathbf{v} = (v_0, v_1, \dots, v_{N/2-1})$ and r is a positive integer.

The operations $+$ and $\times$ represent elementwise addition and multiplication in the plaintext space, respectively. The `depth` parameter specifies the maximum number of sequential multiplications that can be performed while ensuring correct decryption, as each multiplication operation increases ciphertext noise. Exceeding this limit may result in decryption failure. While `depth` is predetermined during parameter selection, `level` dynamically indicates the remaining multiplication capacity, decreasing with each multiplication operation executed.

For practical HE implementations, several general-purpose HE libraries are available, including SEAL-Python [2], Lattigo [38], HELib [18], and OpenFHE [5]. We selected SEAL-Python for its efficient support of the CKKS scheme. Building on this foundation, we developed UniHENN, a HE-based framework specifically optimized for CNN inference. While currently implemented with SEAL-Python, UniHENN is designed to be adaptable to other libraries.

III. OVERVIEW OF UniHENN

In this section, we introduce UniHENN, a HE-based framework designed for inference on encrypted data. UniHENN uses the CKKS HE scheme to encrypt input data, facilitating

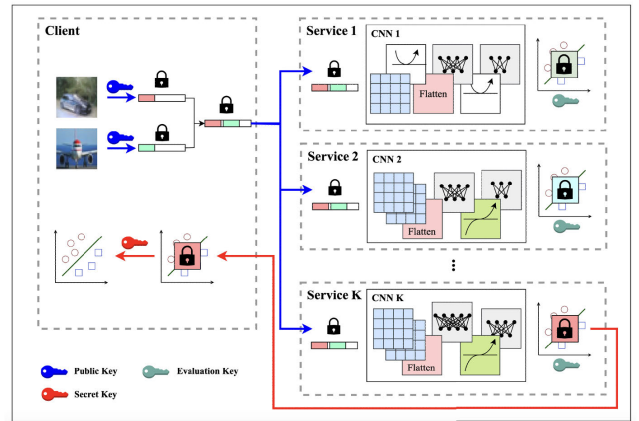


FIGURE 1. Overview of the UniHENN architecture.

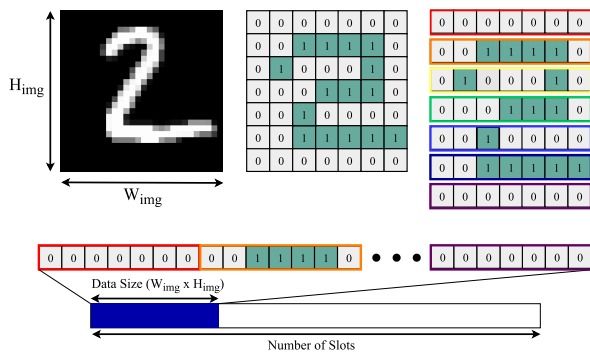
its integration into any CNN models without needing the `im2col` function, which demands a specific input shape. To minimize the computational overhead of HE-based inference, we employ three innovative techniques:

- 1) Unlike previous approaches [7], [15], [18] that rely on the input size, UniHENN calculates the total number of HE operations for the fully connected layer based on the output size. As the output size is typically smaller than the input size in the fully connected layer, this method reduces the average time per operation.
- 2) UniHENN enables batch operations, allowing the consolidation of multiple ciphertexts into one for more efficient processing. This considerably reduces the overall computational time, marking a significant advantage for UniHENN in large-scale data processing scenarios.
- 3) UniHENN opts for average pooling, which eliminates the need for multiplication, thereby reducing operation time. This configuration allows for more layers within given parameter settings, giving service providers more flexibility to incorporate average pooling without concern for operation count.

Figure 1 presents a high-level overview of UniHENN's operational flow. The process begins when a client encrypts one or more images using a public key. These encrypted images are consolidated into a single ciphertext and sent to a cloud service specializing in data analytics. Each service (i.e., Service 1, Service 2, ..., Service K) uses its unique CNN model for data processing, executing specific algorithms and computations on the received ciphertext with the evaluation key. Each CNN model has a distinct layer architecture and optimized parameters. After processing the ciphertext through their respective CNN models, the encrypted inference results are returned to the client. The client decrypts these results using the corresponding secret key to obtain the processed outcomes from each CNN model. This framework allows the client to benefit from multiple CNN models while preserving the confidentiality of the data, indicating a significant step forward in privacy-preserving

TABLE 1. Notations used for UniHENN.

Notation	Definition
N	Degree of polynomial. It is in the form of a power of two.
D, L	Depth and level of ciphertext. This represents the current remaining limit on the number of multiplication operations allowed for the ciphertext.
CH_{in}, CH_{out}	Number of input and output channels in layer
DAT_{in}, DAT_{out}	Number of input and output data in FC layer
R_q	Polynomial quotient ring ($\mathbb{Z}_q / \langle X^N + 1 \rangle$). It is used to create a vector space of the ciphertext.
W_{img}, H_{img}	Width and height of the input image data
K	Width and height of the kernel
W_{in}, H_{in}	Width and height of input data from this layer
W_{out}, H_{out}	Width and height of output data from this layer
S_{total}	It is used in Section IV-E. The value is the multiplication of each convolutional layer's stride and the kernel sizes of all average pooling layers.

**FIGURE 2.** Data transform for UniHENN.

machine learning. Table 1 summarizes the notations used throughout this paper.

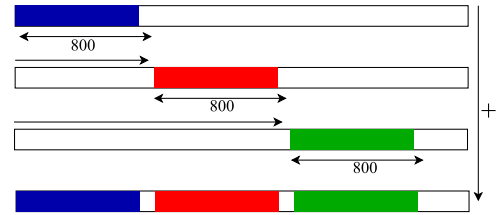
A. CONSTRUCTION OF INPUT DATA

There is a variety of data types for ML services. For instance, while image data is two or three-dimensional, statistical data is usually one or two-dimensional. To handle such varying data dimensions, UniHENN initially flattens the input data into a one-dimensional array in row-major format. When performing encryption, the data is located from the first of the list, and the remaining space is filled with zero to perform encryption. The remaining space will be used in the batch operations.

Figure 2 shows an example of data transformation for encryption. The input data consists of $W_{img} \times H_{img}$ numbers. We flatten the original data row by row, as shown in Figure 2. This flattening procedure ensures universal compatibility of the input data with ML models, enhancing adaptability to algorithms used by ML services.

B. COMBINING CIPHERTEXTS FOR BATCH OPERATION

The technique in UniHENN combines multiple encrypted input data into a single ciphertext for batch processing. Incorporating multiple encrypted data into a single ciphertext

**FIGURE 3.** Combining multiple ciphertexts into one.

involves sequentially inserting each encrypted data, ensuring enough space to prevent overlap with other encrypted data, as shown in Figure 3. This is achieved by rotating each encrypted data before adding it to the ciphertext.

Importantly, the size allocation is not solely based on the input size of the encrypted data but also considers the size of the intermediate or final output vector generated by performing CNN operations. By examining the given CNN model structure, we can pre-calculate this size in advance, thereby preventing the overlap of input data results.

Figure 3 illustrates one example. In this case, 800 slots are needed to process the input data through the CNN model. Thus, the i -th input data would be rotated to the right by $(i - 1) \times 800$ slots, and the rotated vector is added to the ciphertext. UniHENN can integrate any number of ciphertexts into a single one, provided the total size of the encrypted data does not exceed the number of slots. This method enhances the efficiency of UniHENN when handling multiple ciphertexts concurrently.

IV. CNN MODEL CONSTRUCTION

A. CONSTRUCTION OF THE DROP-DEPTH

The parameter is pre-defined because UniHENN's design philosophy is to support the ciphertext that is encrypted before the model is fixed. So, the CKKS parameter is determined and encrypted before the CNN model is defined for reusability of the input ciphertext independent of the model. But, the multiplication and rotation time of HE is increased dependent on the *level*. Thus, the best choice of the CKKS parameter is determined by the number and structure of the CNN model's layers. However, in this scenario, the *depth* is defined before when the CNN model is fixed, which means that the *depth* can be larger than the exact number of total levels of the CNN's model. Thus, we suggest the *Drop-Level*, a novel approach to reduce the *level* that is optimized when the target CNN model. If the input ciphertext has *depth* D and the target CNN model's level is L where $L \leq D$ then the *Drop-Level* reduces the *level* of the input ciphertext as L with multiply $D - L$ ciphertexts that are the encryption of vectors where all elements are consists of 1. By these simple structures, the time of execution is very fast (In our experiments in Section VII, the time of *Drop-Level* operation does not exceed 100ms).

B. CONSTRUCTION OF THE CONVOLUTIONAL LAYER

The `im2col` encoding is a popular and efficient algorithm for transforming multidimensional data into matrix form to

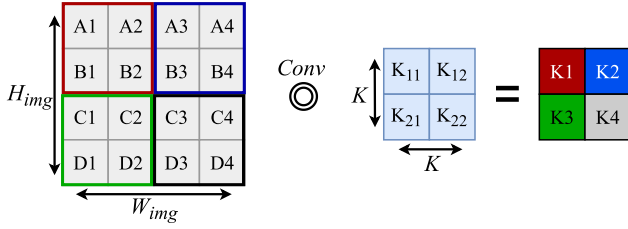


FIGURE 4. Convolution operations on the two-dimensional input data and kernel.

facilitate convolution operations. However, it requires precise arrangement of input matrix elements, which is challenging to achieve on ciphertexts. Therefore, TenSEAL [7], a sophisticated open-source HE-enabled ML SDK, performs this operation in a preprocessing step before encryption, allowing only a single convolutional layer.

To simplify this process and improve flexibility, we propose a new method for constructing convolutional layers without relying on im2col. Our approach uses flattened input data, eliminating the need for a specific arrangement of input matrix elements as follows:

- 1) **Kernel and Stride:** In convolutional layers, a small matrix called a kernel moves over the input image. Each movement of the kernel is called a stride. At each position, the kernel multiplies its values with the corresponding values in the input image and sums the results.
- 2) **Input and Kernel Dimensions:** The dimensions of the input image are denoted as (W_{img}, H_{img}) , with a stride of S . For simplicity, we assume the kernel has equal width and height, both represented by K .
- 3) **Flattening the Input:** In the context of HE, we flatten the two-dimensional input data into a one-dimensional form. This simplifies the convolution process under encryption.
- 4) **Rotation and Multiplication:** The convolution operation involves rotating the input data and multiplying it with the kernel. The number of rotations equals the number of elements in the kernel. Each element in the input data is multiplied by the corresponding element in the kernel, and the results are summed up.
- 5) **Output:** The output of the convolution is an array with values spaced according to the stride interval.

Figures 4 and 5 illustrate the convolution operations on two-dimensional and flattened input data, respectively.

Figure 5 illustrates the convolution process: the input data is incrementally shifted by S , K times, with leftward rotations after each shift. These rotated ciphertexts are then multiplied by a specially structured kernel vector containing a single kernel element, effectively eliminating unrelated elements during convolution. Algorithm 1 provides a detailed description of this convolution layer construction.

This approach improves the convolution process by decoupling the number of operations from the image size and limiting rotations to the square of the kernel size K .

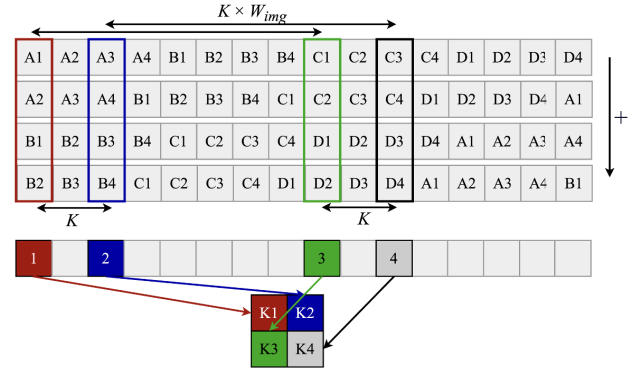


FIGURE 5. Convolution operations on the flattened data.

Algorithm 1 Construction of the Convolution Layer

Input:

- List of Ciphertext: $C_{in} := (C_{in(0)}, C_{in(1)}, \dots, C_{in(CH_{in}-1)}) \in (\mathcal{R}_q^2)^{CH_{in}}$
- Kernel: $Ker := (Ker_{(o,i)})_{0 \leq o < CH_{out}, 0 \leq i < CH_{in}} \in \mathbb{R}^{CH_{out} \times CH_{in} \times K \times K}$
- Bias: $B := (B_{(o)})_{0 \leq o < CH_{out}} \in \mathbb{R}^{CH_{out}}$
- Interval: I_{in}
- Stride: S

Output:

- List of Ciphertexts: $C_{out} \in (\mathcal{R}_q^2)^{CH_{out}}$
- Interval: I_{out}

Procedure: $*$ (All ciphertexts and operations are in $\mathcal{R}_q^2$)

```

for  $i = 0$  to  $CH_{in} - 1$  do
  for  $j = 0$  to  $K - 1$  do
    for  $k = 0$  to  $K - 1$  do
       $C_{rot(i,j,k)} \leftarrow Rot(C_{in(i)}, I_{in} \times (k + W_{img} \times j))$ 
    end for
  end for
end for
for  $o = 0$  to  $CH_{out} - 1$  do
   $C_{out(o)} \leftarrow \sum_{i=0}^{CH_{in}-1} \sum_{j=0}^{K-1} \sum_{k=0}^{K-1} Mul(C_{rot(i,j,k)}, K_{(o,i,j,k)})$ 
   $C_{out(o)} \leftarrow Add(C_{out(o)}, B_{(o)})$ 
end for
 $C_{out} \leftarrow (C_{out(0)}, \dots, C_{out(CH_{out}-1)})$ 
 $I_{out} \leftarrow I_{in} \times S$ 
return  $C_{out}, I_{out}$ 

```

The number of plaintext multiplications in the convolution layer of our method is $\mathcal{O}(CH_{in} \cdot CH_{out} \cdot K^2)$ and the number of rotations is $\mathcal{O}(CH_{in} \cdot K^2)$. Considering the complexity of plaintext multiplication $\mathcal{O}(N \cdot L)$ and rotation $\mathcal{O}(N \cdot \log N \cdot L^2)$ as stated in [31], the overall complexity of the convolutional layer is $\mathcal{O}(N \cdot L \cdot CH_{in} \cdot K^2 \cdot (CH_{out} + \log N \cdot L))$.

C. CONSTRUCTION OF THE AVERAGE POOLING LAYER

CNNs use pooling layers to reduce input size, with max, min, and average pooling being common. Our architecture prioritizes high-speed CNN inference without bootstrapping.

Ablation studies on a LeNet-5-like model revealed comparable accuracies for min (0.989), max (0.992), and average (0.985) pooling. In HE, max and min pooling require numerous multiplications [33], necessitating costly bootstrapping, while average pooling needs only one multiplication. Given the negligible performance difference and significant computational savings, we implemented average pooling in UniHENN.

We optimize the average pooling layer by eliminating the constant multiplication by $1/c^2$ (where c is the kernel size). Instead, we apply this multiplication in a preceding convolutional or flatten layer. This approach maintains functionality while reducing the number of multiplications and lowering depth, enhancing overall efficiency.

The implementation depends on the subsequent layer. If followed by an activation layer, the constant multiplication is moved to the preceding flatten or convolutional layer. For a linear layer $h(x) = Ax + b$, we use $h(x/c^2) = (1/c^2)Ax + b$, incorporating the scaling factor into the weight matrix. This strategy preserves the average pooling effect while optimizing computational resources.

When an activation function such as the square function or the approximate ReLU function follows average pooling, we cannot apply the above logic directly because these functions are not linear. In this case, we can apply the following logic:

- If the activation function is the square function: Then, $(1/c^2)^2 = 1/c^4$ is applied the following in the convolutional layer or the flatten layer.
- If the activation function is the approximate ReLU function: Then, we can apply the coefficient of the approximate ReLU $f(x/c^2) = 0.375373 + (0.5/c^2)x + (0.117071/c^4)x^2$ if the approximate ReLU is defined as $f(x) = 0.375373 + 0.5x + 0.117071x^2$.

The mechanism of average pooling closely resembles that of the convolutional layer. Suppose an average pooling operation is conducted with a kernel size of c . We can then apply a convolutional layer with c as both the kernel size and stride and use a constant multiplication of $1/c^2$ as described. The overview of this logic is shown in Figure 5.

However, unlike the convolutional layer, *invalid values* are introduced in the average pooling layer because the flattened kernels are not multiplied, as illustrated in Figure 6. Due to this issue, the gap between data requires maximum rotation to the left. The exact interval is as follows:

$$(W_{img} + 1) \times (c - 1) \quad (1)$$

D. CONSTRUCTION OF THE FULLY CONNECTED (FC) LAYER

In an FC layer, the input data is multiplied by a weights matrix, as depicted in Figure 7 (a). PycrCNN performs the same procedure as a plain FC layer by multiplying encrypted data with the weights matrix. However, in this case, matrix multiplication requires significant computational power and time.

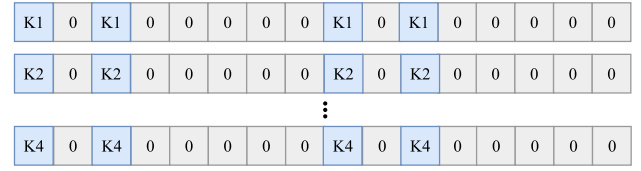


FIGURE 6. Flattening a two-dimensional kernel into a one-dimensional array.

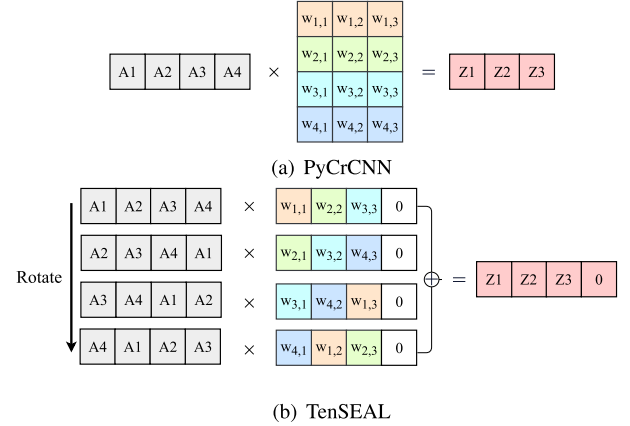


FIGURE 7. Structure of the FC layer of (a) PyCrCNN and (b) TenSEAL.

Employing the vector multiplication technique [18] offers a way to facilitate FC layer operations on the ciphertext. As depicted in Figure 7 (b), TenSEAL employs the diagonal method for vector-matrix multiplication [7], [18]. In this approach, rotated data is multiplied by a rotated weights vector. To utilize vector multiplication, the width of the weight matrix is set to the input size by padding with zeros. Consequently, the number of multiplications and rotations depends on the FC layer's input size. Generally, the input size of the FC layer is larger than its output size, which may consume unnecessary resources. To minimize the number of resource-intensive operations, including multiplication and rotation, we optimized the logic to depend on the FC layer's output size.

In Figure 8, the overall process of vector multiplication in UniHENN is illustrated. First, as shown in Figure 8 (a), pad zeros until the number of rows is a multiple of the output size, and then rotate the columns of the weight matrix upward, incrementing one column at a time as shown in Figure 8 (b). Next, append the segments of each array with a negative index to the end of the array. Finally, as depicted in Figure 8 (c), divide the matrix lengthwise by the output size and concatenate the parts horizontally; the weight matrix M_w then becomes as shown in Figure 8 (d). Note that generating an optimized weight matrix (from Figure 8 (a) to Figure 8 (d)) involves modifying the weights in plaintext; however, the operation time is negligible.

In the inference process, the input ciphertext is rotated and multiplied by the weight matrix M_w , as depicted in Figure 8 (e). Matrix multiplication can be performed in

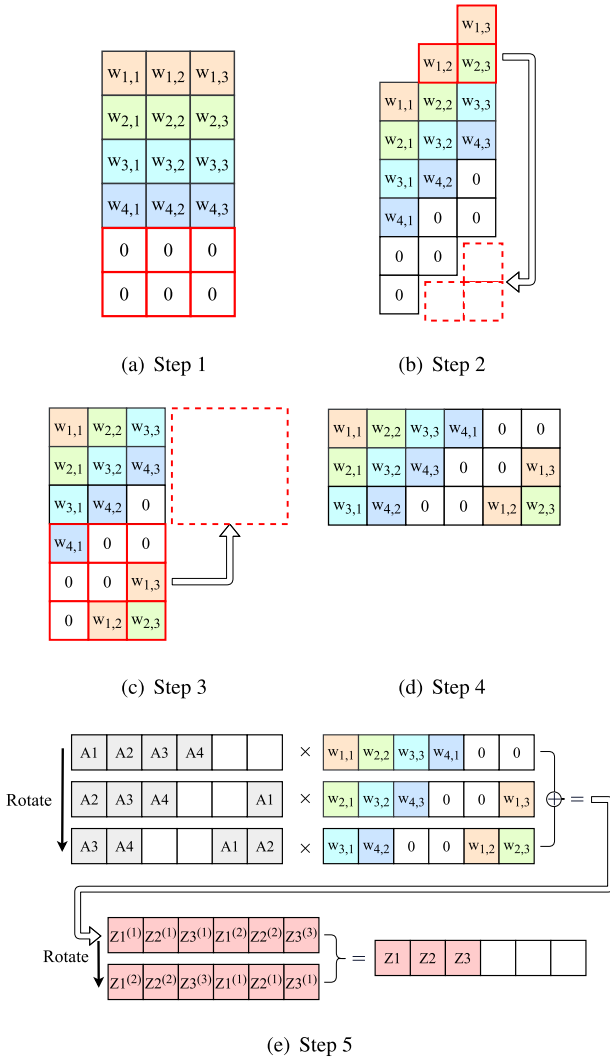


FIGURE 8. Construction of FC layer of UniHENN.

encrypted form with only multiplication operations that depend on the output size. Generally, since the output size of the FC layer is smaller than the input size, operations can be more efficient than with the diagonal method. For example, if the FC layer has an input size of 64 and an output size of 10, then the total number of multiplications and rotations would be 10, not 64.

The number of plaintext multiplications of the FC layer in UniHENN is $\mathcal{O}(DAT_{out})$, and the number of rotations is $\mathcal{O}(DAT_{in}/DAT_{out} + DAT_{out})$. This means that if DAT_{in} is smaller than the square of DAT_{out} , it can be denoted as $\mathcal{O}(DAT_{out})$. In this case, the time complexity of the FC layer depends only on the output size. Therefore, the complexity of the FC layer in UniHENN is $\mathcal{O}((DAT_{in}/DAT_{out} + DAT_{out}) \cdot N \cdot \log N \cdot L^2)$.

E. CONSTRUCTION OF THE FLATTEN LAYER

The outputs of the convolutional layer (or average pooling layer) typically have an interval (the reason is detailed in

Algorithm 2 Construction of the Fully Connected Layer

Input:

- Ciphertext: $C_{in} \in \mathcal{R}_q^2$
- Weight matrix: $M_w \in \mathbb{R}^{DAT_{out} \times DAT_{in}}$
- Bias: $B \in \mathbb{R}^{DAT_{out}}$

Output:

- Ciphertext: $C_{out} \in \mathcal{R}_q^2$

Procedure: *

 (All ciphertexts and operations are in $\mathcal{R}_q^2$)

$M_{rot} \leftarrow$ Apply M_w from step 1 to 4 in Figure 8

$W_{vec} = \lceil DAT_{in}/DAT_{out} \rceil \times DAT_{out}$

$C_{sum} \leftarrow C(0)$

for $o = 0$ to $CH_{out} - 1$ **do**

$V_1 \leftarrow M_{rot(o)}[W_{vec} - o] + [0] \times o$

$C_{sum} \leftarrow Add(C_{sum}, Mul(Rot(C_{in}, o), V_1))$

$V_2 \leftarrow [0] \times (W_{vec} - o) + M_{rot(o)}[W_{vec} - o]$

$C_{sum} \leftarrow Add(C_{sum}, Mul(Rot(C_{in}, o - W_{vec}), V_2))$

end for

$C_{out} \leftarrow C(0)$

for $i = 0$ to $\lceil DAT_{in}/DAT_{out} \rceil - 1$ **do**

$C_{out} \leftarrow Add(C_{out}, Rot(C_{sum}, i \times DAT_{out}))$

end for

$C_{out} \leftarrow Add(C_{out}, B)$

return C_{out}

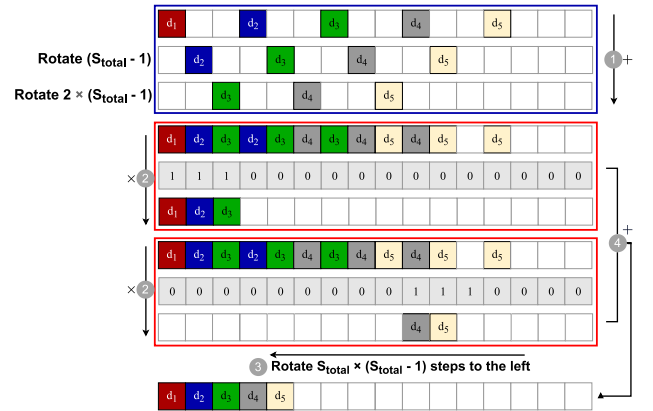


FIGURE 9. Removing row interval when the previous layer is either a convolutional layer or an approximate ReLU layer.

Section IV-B). Additionally, the output of the convolutional layer is usually sparse, as the output from each filter resides in a distinct ciphertext. In a flatten layer, the operation is performed to collect each ciphertext and integrate it into a single ciphertext. Sparse ciphertext leads to unnecessary time and memory consumption. To avoid this, we construct flatten layers to remove invalid data between valid data and to compress all convolutional computation results into a single kernel ciphertext.

1) REMOVING THE ROW INTERVAL

After passing through the convolution layer or fully connected layer, the valid data in the output ciphertext will be offset, which we call row interval. We propose an algorithm to remove the row interval for two situations: whether the

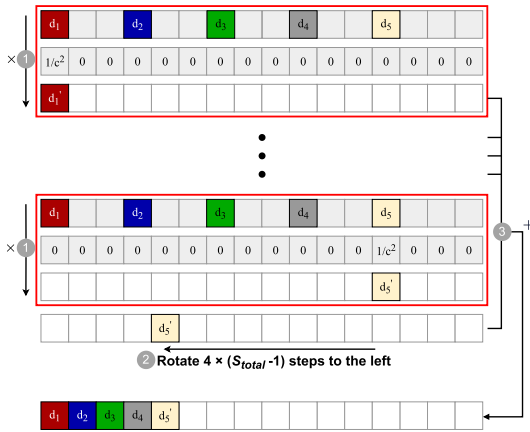


FIGURE 10. Removing invalid data between valid data when the previous layer is an average pooling layer.

preceding layer is an average pooling layer or not. As detailed in Section IV-C, the square layer does not handle constant multiplication, so it is not considered when identifying the previous layer.

If the previous layer is either a convolutional layer or an approximate ReLU layer, it may contain some zero values between each set of non-zero values. We can exploit this to efficiently execute the flatten operation by stacking the non-zero values using rotation and addition.

Additionally, if $W_{in} = 1$, there is only one column. Therefore, this removing row interval step can be skipped.

- In step ①, perform $(S_{total} - 1)$ rotations $(S_{total} - 1)$ times. Accumulate all the vectors produced from each rotation.
- In step ②, a vector containing S_{total} ones and $(W_{img} - 1) \times S_{total}$ zeros is rotated in vector and multiplied $\lceil W_{in}/S_{total} \rceil$ times with the ciphertexts, allowing the extraction of S data points at once.
- In step ③, after multiplications, the i th sparse vector is rotated $(i - 1) \times S_{total} \times (S_{total} - 1)$ times to the left for $i \in [2, \lceil W_{in}/S_{total} \rceil]$.
- In step ④, all rotated vectors are added.

2) REMOVING INVALID DATA AND ROW INTERVAL

If the previous layer is an average pooling layer, *invalid values* exist in the intervals between each set of valid data points. Moreover, the required constant multiplication (assuming a factor of $1/c^2$) has not been applied during the average pooling operation. Consequently, we must eliminate these invalid data points and perform the necessary multiplication. To address this issue, we suggest a three-step process as follows.

Figure 10 ① illustrates the flatten layer's operational algorithm to remove invalid data. Unlike Figure 9, some invalid values exist instead of the zero values when a flatten layer comes after an average pooling layer. Therefore, we can't operate data simultaneously. Thus, each value is needed to be calculated one by one.

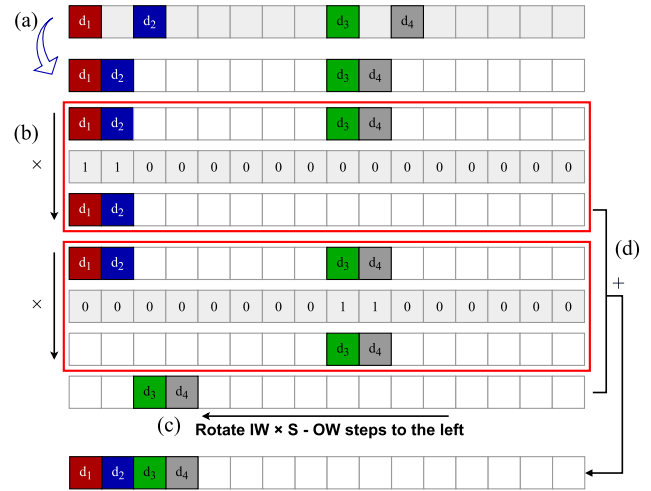


FIGURE 11. Overall mechanism for compressing all convolutional computation results into a single kernel ciphertext.

Suppose S_{total} is the value obtained by multiplying the stride values of all convolutional layers by the kernel sizes of all average pooling layers, $W_{img} \times H_{img}$ is the image size and $W_{in} \times H_{in}$ is the output size of the last average pooling layer.

- In step ①, a vector containing one $1/c^2$ and $(W_{img} \times S_{total} - 1)$ zeros is rotated in vector and multiplied W_{in} times with the ciphertexts.
- In step ②, after multiplication, the i -th sparse vector is rotated $(i - 1) \times (S_{total} - 1)$ times to the left for $i \in [2, W_{in}]$.
- In step ③, all rotated vectors are added.

3) REMOVING THE COLUMN INTERVAL

As shown in Figure 11, the overall computation mechanism within the HE-based convolutional layer comprises four steps. After removing the row interval (step ①), it is apparent that the column interval still exists. For models utilizing 1D CNN or 2D CNN inference with $H_{in} = 1$, this removing column interval step can be skipped due to the presence of only one row. However, for 2D CNN models, an additional set of 3 steps is executed to remove the column interval, as indicated in Figure 11.

- In step ②, a vector containing W_{img} ones and the rest as zeros is rotated and multiplied with the ciphertexts.
- In step ③, after the multiplication, rotate the i -th sparse vector $(i - 1) \times (W_{img} \times S_{total} - W_{in})$ times to the left, where $i \in [2, H_{in}]$.
- In step ④, sum up all the rotated vectors.

These additional steps help to remove the column intervals effectively, ensuring that the data is adequately flattened and ready for further processing in the CNN pipeline.

4) PACKING MULTIPLE CIPHERTEXTS

Figure 12 shows the detail of the steps taken to combine the multiple output vectors produced by the convolution

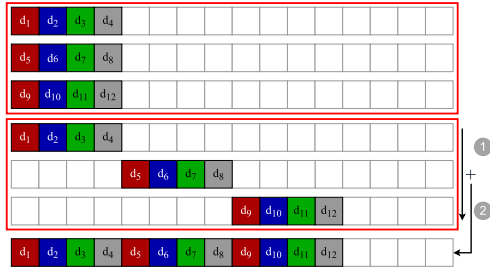


FIGURE 12. Overall mechanism for multiple ciphertexts into a single ciphertext.

operation into a singular ciphertext. The descriptions are as follows:

- In step ①, each of the output vectors is right-rotated $W_{in} \times H_{in}$ steps.
- In step ②, all these rotated vectors are aggregated together. This aggregation step is particularly efficient because it avoids the need for multiplication operations, which are computationally intensive when applied to ciphertexts. This efficiency is due to the sparsity of the vector.

After completing this procedure, the outputs are flattened into a single ciphertext. The number of slots occupied in this ciphertext will be $W_{in} \times H_{in} \times CH_{in}$.

The convolutional layer will also require two multiplication operations when the stride S exceeds one. However, if S_{total} equals one, the process illustrated in Figure 11 (a) becomes unnecessary. Thus, only a single multiplication operation would be needed.

5) TIME COMPLEXITY

The number of plaintext multiplications in the flatten layer of UniHENN is $\mathcal{O}(W_{in} + H_{in})$ and the number of rotations in the flatten layer of UniHENN is $\mathcal{O}(CH_{in} \cdot S_{total} + W_{in} + H_{in})$.

Therefore the complexity of flatten layer of UniHENN is $\mathcal{O}((CH_{in} \cdot S_{total} + W_{in} + H_{in}) \cdot N \cdot \log N \cdot L^2)$

F. CONSTRUCTION OF THE ACTIVATION LAYER

Activation functions require many multiplications [33], so it requires many computing resources to implement them with HE. To address this, we approximate each activation function using a low-degree polynomial. For instance, a 2-degree polynomial approximation of the ReLU function [20] is represented as $f(x) = 0.375373 + 0.5x + 0.117071x^2$. In UniHENN, these approximated activation functions are used instead of the real activation functions for efficiency.

V. OPTIMIZING CIPHERTEXT SIZE FOR BATCH PROCESSING

As shown in Figure 3, UniHENN combines multiple encrypted input data into a single ciphertext for batch processing. However, the ciphertext size for each input should not be allocated solely based on the size of the input data. Instead, it should also consider the intermediate or final output values produced by performing CNN operations. This

prevents overlapping issues with other encrypted input data's intermediate or final output values. We can determine this size in advance by examining the structure of the CNN model.

This section will explain how the output size can be determined in each of the layers (convolutional layer, average pooling layer, flatten layer, and fully connected layer) implemented in UniHENN.

A. CONVOLUTIONAL LAYER

The size of the convolution layer's output data depends on three parameters: stride, padding, and kernel size.

- **Stride:** In the convolutional layer of UniHENN, the interval between each row of data equals the product of the strides from all previous layers, while the interval between each column equals this product multiplied by the image width.
- **Padding:** In a convolutional layer with padding, extra space is required to add 0 elements. If there are L convolutional layers with padding spaces $P_1, P_2, \dots, P_L$, then allocate a space of length $P = P_1 + P_2 + \dots + P_L$ in the input ciphertext. This space is used for each padding process. As padding is accounted for in the initial input image size, it is not considered when calculating the largest size in **Theorem 1**.
- **Kernel size:** A larger kernel size results in a smaller output size, as explained in **Theorem 1**.

The output size of the convolutional layer does not always exceed the input size when the kernel is greater than or equal to the stride. This is because the convolutional operation is designed to be calculated independently in each channel. The proof of this statement is described in **Theorem 1**.

In **Theorem 1**, one condition is that the stride value is not larger than the kernel size. This is realistic, as a larger stride value would result in data loss from the image.

Theorem 1: Let $W_{img} \times H_{img}$ be the image size and N_{layer} the number of total convolutional layers. Denote $(W_{in(i)}, H_{in(i)})$ and $(W_{out(i)}, H_{out(i)})$ as the input and output sizes of the i -th convolutional layer, and the kernel size is $(W_{ker(i)}, H_{ker(i)})$, stride size is $(W_{st(i)}, H_{st(i)})$, and padding size is $(W_{pad(i)}, H_{pad(i)})$ for all $i \in [1, N_{layer}]$. Additionally, let

$$H_{[st(i)]} = \prod_{j=1}^i H_{st(j)}$$

Then, $H_{img} \geq H_{out(i)} \times H_{[st(i)]}$, where $H_{ker(i)} \geq H_{st(i)} \geq 1$ for all $i \in [1, N_{layer}]$.

Proof: We obtain that $H_{out(i)}$:

$$H_{out(i)} = \left\lceil \frac{H_{in(i)} - H_{ker(i)}}{H_{st(i)}} \right\rceil + 1$$

Given the hypothesis, the following inequality holds:

$$\begin{aligned} H_{out(i)} \times H_{[st(i)]} &= \left(\left\lceil \frac{H_{in(i)} - H_{ker(i)}}{H_{st(i)}} \right\rceil + 1 \right) \times H_{[st(i)]} \\ &= \left\lceil \frac{H_{in(i)} - H_{ker(i)} + H_{st(i)}}{H_{st(i)}} \right\rceil \times H_{[st(i)]} \end{aligned}$$

$$\leq \left\lceil \frac{H_{in(i)}}{H_{st(i)}} \right\rceil \times H_{st(i)} \leq H_{in(i)} = H_{out(i-1)}$$

When $i > 1$, $H_{in(i)} = H_{out(i-1)}$. As proven by mathematical induction, inserting an average pooling layer or an activation function between convolutional layers does not change the size. $\square$

Note that after passing through the i -th convolutional layer, the interval in each column of data is $W_{img} \times (H_{st(i)} - 1)$ (detailed in IV-B). Given W_{img} columns of data or fewer, the image size of $H_{img} \times W_{img}$ is not exceeded by **Theorem 1**.

Corollary 1.1: Denote

$$W_{[st(i)]} = \prod_{j=1}^i W_{st(j)}$$

Then, $W_{img} \geq W_{out(i)} \times W_{[st(i)]}$, where $W_{ker(i)} \geq W_{st(i)} \geq 1$ for all $i \in [1, N_{layer}]$.

Proof: The proof is analogous to **Theorem 1** and is therefore omitted. $\square$

We adopt a slot-based operation approach, allowing data with remaining intervals to pass through to the next layer unchanged. The *row interval* is defined as the length between row vectors, and the *column interval* is the length between column vectors. Specifically, if the stride of the i -th convolutional layer is S_i , the interval between each data value after this layer is:

$$\text{row interval} : \prod_{j=1}^i S_j - 1$$

$$\text{column interval} : W_{img} \times \left(\prod_{j=1}^i S_j - 1 \right)$$

By **Theorem 1** and **Corollary 1.1**, each data value stays within the specified index, confirming the flawless operation of the convolutional layer in UniHENN.

B. AVERAGE POOLING LAYER

Assuming that the kernel size in the average pooling layer is c , each channel uses a filter with a kernel size of (c, c) and a stride of (c, c) . The result is then multiplied by $1/c^2$. We omit the multiplication of a vector consisting of 0s and 1s, allowing some non-zero but unused values (i.e., *invalid values*) to overlap with other values. According to **Theorem 1**, the number of slots in the ciphertext data remains unchanged when passing through UniHENN's average pooling. However, to prevent *invalid values* from occupying the positions of other used data, an additional space of $(W_{img} + 1) \times (c - 1)$ is required (detailed in IV-C).

C. FLATTEN LAYER

Assume that we flatten data from CH_{in} channels, each of size $W_{in} \times H_{in}$. After flattening, the ciphertext data will have $W_{in} \times H_{in} \times CH_{in}$ slots. This size can exceed the maximum size of the layers preceding the flatten layer. We compare these two sizes to determine the largest possible size of the

hidden layer. Because the model has plaintext information, comparison operations are possible.

D. FULLY CONNECTED LAYER

Let CH_{in} and CH_{out} be the input size and output size of the FC layer, respectively. The size of the slot required in the FC layer is given by:

$$CH_{out} \times \left\lceil \frac{CH_{in}}{CH_{out}} \right\rceil$$

Details of how this formula was derived can be found in Section IV-D. The maximum size up to the preceding layer is compared with this value and updated to the larger of the two.

VI. EXPERIMENTS

This section presents experimental results demonstrating the feasibility and effectiveness of UniHENN.

We implement seven distinct CNN models, including LeNet-1 [28] and LeNet-5 [30], to demonstrate that UniHENN can be applicable to a wide range of model architectures.

A. EXPERIMENTAL SETTINGS

We used a server on the NAVER Cloud platform [42] with the following specifications: 16 vCPU cores (Intel(R) Xeon(R) Gold 5220 CPU @ 2.20GHz), 64GB of memory, and a 50GB SSD.

B. DATASETS

We utilized four different datasets for our models: MNIST [14], CIFAR-10 [27], USPS [19], and ECG [37].

1) MNIST DATASET

The MNIST dataset [14] consists of 70,000 grayscale images of handwritten digits, each sized 28×28 pixels, with 10 classes representing the digits from 0 to 9. We used 60,000 images for training and 10,000 images for testing.

2) CIFAR-10 DATASET

The CIFAR-10 dataset [27] includes 60,000 RGB images, each sized 32×32 pixels, with 10 classes. We used 50,000 images for training and 10,000 images for testing.

3) USPS DATASET

The USPS dataset [19] contains 9,298 grayscale images of handwritten digits, each sized 16×16 pixels, with 10 classes. We used 7,291 images for training and 2,007 images for testing.

4) ECG DATASET

The ECG dataset from the MIT-BIH arrhythmia database [37] consists of 109,446 samples at a sampling frequency of 125 Hz. It includes signals representing ECG shapes of heartbeats in normal cases and various arrhythmias.

TABLE 2. Parameters used in UniHENN. *PK*: public key size (encryption key), *SK*: secret key size (decryption key), *GK*: Galois key size, *RK*: Relinearization keys size. *GK* and *RK* together form the evaluation key. # mult: total allowed multiplications.

# slots	8,192
scale factor	32
log Q	432
<i>PK</i> (MB)	1.87
<i>SK</i> (MB)	0.94
<i>GK</i> (GB)	0.57
<i>RK</i> (MB)	22.52
<i>ctxt</i> (MB)	1.68
# mult (<i>depth</i>)	11

Following the preprocessing steps by Abuadbba et al. [4], we used a subset of 26,490 samples, with 13,245 samples for training and 13,245 samples for testing, available at <https://github.com/SharifAbuadbba/split-learning-1D>.

C. HYPERPARAMETER SETTINGS FOR TRAINING

We trained all models with a learning rate of 0.001 and a batch size of 32. The MNIST and USPS models were trained for 15 epochs, the CIFAR-10 model for 25 epochs, and the ECG model for 30 epochs, based on their convergence behavior.

D. LIBRARY AND PARAMETER SETUP

We used SEAL-Python [2] for homomorphic operations, ensuring consistency with our baseline models TenSEAL and PyCrCNN, which are also SEAL-based. PyTorch and torchvision were used for model training. Detailed information about libraries and the experimental setup is available at our GitHub repository: <https://github.com/hm-choi/uni-henn>. Table 2 provides the specific parameters used in our experiments. All security parameters comply with the 128-bit security level as described in [43].

VII. EXPERIMENTAL RESULTS

In this section, we perform various experiments to verify the feasibility and evaluate the performance of UniHENN. Each experiment in this paper was conducted under identical conditions and repeated 30 times to ensure consistency. The objectives of our experiments are as follows:

- **Comparison of Inference Time with State-of-the-Art Solutions.** In this section, we compare the inference performance of UniHENN against two state-of-the-art HE-based deep learning inference frameworks: TenSEAL [7] and PyCrCNN [15]. We selected TenSEAL for comparison because it is a well-known open-source library that uses the `im2col` algorithm to implement CNNs. This algorithm allows TenSEAL to achieve highly optimized and efficient inference times for convolution operations. However, this efficiency comes at the cost of flexibility: TenSEAL's architecture only supports models with a single convolutional layer, making it unsuitable for CNN architectures with multiple convolutional layers.

PyCrCNN, on the other hand, does not employ the `im2col` algorithm, making it applicable to a broader range of CNN architectures. However, this flexibility results in slower inference times compared to TenSEAL. Our experiments show that UniHENN successfully balances both flexibility and efficiency. It achieves inference times comparable to those of TenSEAL while supporting CNN architectures with multiple convolutional layers, making UniHENN a more versatile solution for HE-based deep learning inference.

- **Adaptability of UniHENN for Various CNN Model Architectures.** One of the compelling features of UniHENN is its ability to adapt to a variety of CNN architectures, including both complex and 1D CNN models. To demonstrate this adaptability, we conducted experiments with several CNN models, including LeNet-5, a seminal CNN model widely adopted for digit recognition tasks [35], [52]. LeNet-5 is more complex than many other models, with multiple convolutional layers and fully connected layers. Our successful implementation of LeNet-5 in UniHENN highlights the system's ability to handle large and complex CNN models effectively. We also considered a 1D CNN architecture particularly useful for sequence-based tasks such as time-series analysis, natural language processing, and certain bioinformatics applications. This capability sets UniHENN apart from solutions like TenSEAL, which is constrained to supporting only single convolutional layer models and does not offer support for 1D CNNs. Through these experiments, we aim to demonstrate UniHENN's comprehensive applicability and its ability to adapt to various CNN architectures, making it a versatile tool for secure and efficient deep learning inference across diverse application domains.
- **Adaptability of UniHENN with Various Datasets.** One of the key strengths of UniHENN is its adaptability to various kinds of data. To demonstrate this, we conducted experiments using four diverse datasets: MNIST, CIFAR-10, USPS, and ECG. MNIST and CIFAR-10 are widely used image classification datasets that serve as standard benchmarks in the deep learning community. MNIST consists of grayscale images of handwritten digits, while CIFAR-10 comprises coloured images spanning ten different object classes. USPS is another grayscale image dataset used for handwriting recognition. ECG represents electrocardiogram data and is commonly used in healthcare applications for diagnosing various heart conditions. By demonstrating that UniHENN performs well across these varied datasets, we aim to show its wide applicability to tasks involving images of different sizes and complexities, as well as specialized domains like healthcare.

A. EXPERIMENT 1: COMPARISON OF INFERENCE TIME BETWEEN UNIHENN, TENSEAL, AND PYCRCNN

In this experiment, we implement the model, denoted as M_1 , using UniHENN, TenSEAL, and PyCrCNN. The M_1 model

TABLE 3. Detailed parameters for M_1 .

Layer	Parameter	# Mult
Conv2d	$CH_{in} = 1, CH_{out} = 8, K = 4, S = 3$	1
Square	-	1
Flatten	-	2
FC1	$DAT_{in} = 648, DAT_{out} = 64$	1
Square	-	1
FC2	$DAT_{in} = 64, DAT_{out} = 10$	1
Total #Mult	-	7

was introduced in the TenSEAL paper [7] and is well-suited for implementation using TenSEAL. Despite its simple architecture, which contains only one convolutional layer, the model achieves a high accuracy of 97.65%. Table 3 provides detailed specifications for this model architecture. The Conv2D parameters – CH_{in} , CH_{out} , K , and S – represent the number of input channels, the number of output channels, kernel size, and stride for a two-dimensional convolutional layer, respectively. Similarly, DAT_{in} and DAT_{out} in the FC layer indicate the input and output dimensions. We use these notations to represent all other remaining models presented in this paper.

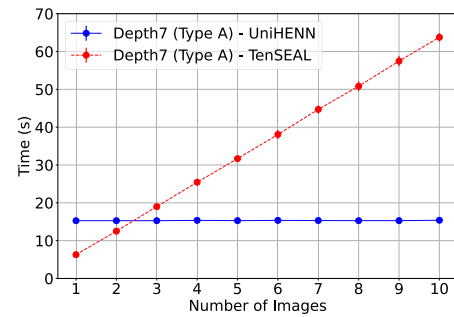
Table 4 presents the results. In terms of inference time for a single sample, the TenSEAL implementation exhibited the highest performance, taking 6.298 seconds. TenSEAL achieved this efficiency by optimizing convolution operations on the input data using the `im2col` algorithm. UniHENN took 16.247 seconds, making it 2.6 times slower than TenSEAL. PyCrCNN was the slowest, requiring 154.494 seconds. However, UniHENN has the advantage when handling multiple data points simultaneously. It outperforms TenSEAL by supporting batch operations for up to ten input samples, allowing concurrent inference calculations. TenSEAL and PyCrCNN do not offer parallel processing for multiple samples, resulting in total times that increase proportionally with the number of input samples. To process ten input samples together, UniHENN still takes 16.247 seconds, approximately 3.9 times and 94.6 times faster than TenSEAL's 63.706 seconds and PyCrCNN's 1537.227 seconds, respectively.

We conducted additional experiments to determine the number of samples at which UniHENN starts to outperform TenSEAL. The experiment was carried out by incrementally increasing the number of samples and observing the inference time. The results are presented in Figure 13.

In Figure 13, the inference times for UniHENN and TenSEAL using the M_1 model are presented. Since TenSEAL does not support batched inference, its inference time increases linearly as the number of input images grows. In contrast, UniHENN does support batched inference, allowing up to 10 MNIST images to be processed simultaneously in the M_1 model. The figure reveals that UniHENN's inference time becomes shorter than TenSEAL's starting at a batch size of 3. This demonstrates that UniHENN surpasses TenSEAL in efficiency when concurrently processing k images, where $k \geq 3$.

TABLE 4. Comparison of the average (with standard deviation) inference time in seconds on the MNIST dataset between PyCrCNN and UniHENN for the M_1 model architecture.

Layer	TenSEAL		PyCrCNN		UniHENN
# of samples	1	10	1	10	1 & 10
Drop Level	-	-	-	-	0.065 (0.002)
Conv2d	2.522 (0.041)	25.202 (0.121)	31.844 (0.483)	316.536 (0.847)	3.436 (0.036)
Square	0.024 (0.001)	0.242 (0.003)	25.082 (0.548)	247.291 (0.844)	0.281 (0.009)
Flatten	-	-	0.000 (0.000)	0.000 (0.000)	5.285 (0.060)
FC1	3.540 (0.293)	36.147 (1.084)	94.883 (0.287)	946.841 (1.407)	6.783 (0.064)
Square	0.014 (0.001)	0.140 (0.003)	1.527 (0.030)	15.017 (0.083)	0.011 (0.000)
FC2	0.198 (0.027)	1.975 (0.077)	1.158 (0.025)	11.542 (0.062)	0.386 (0.009)
Total	6.298 (0.291)	63.706 (1.113)	154.494 (1.110)	1537.227 (2.555)	16.247 (0.103)

**FIGURE 13.** Average inference time (in seconds) for the M_1 model architecture using UniHENN and TenSEAL with varying the number of input samples.

B. EXPERIMENT 2: COMPARISON OF INFERENCE TIME BETWEEN UNIHENN AND PYCRCNN FOR LENET-1

In this experiment, we utilize the LeNet-1 model, denoted as M_2 . The accuracy of M_2 is 98.62%, slightly higher than M_1 . The detailed specifications of these model architectures can be found in Table 5. Implementing the hyperbolic tangent function ($\tanh$) in HE is computationally challenging; therefore, we have substituted the activation function from $\tanh$ to the square function. We find that the modified LeNet-1 model with the square activation achieves an accuracy of 98.62% comparable to the 98.41% accuracy of the original LeNet-1 model when tested on a 10,000 sample MNIST dataset. This indicates that the modification in the activation function does not significantly impact the model's accuracy.

This experiment aims to confirm that UniHENN can support CNN models with more than two convolutional layers. Importantly, the TenSEAL library cannot be used in this experiment due to its constraint of supporting only a single convolutional layer, a limitation arising from its `im2col` algorithm implementation. The results are presented in Table 6.

TABLE 5. Detailed parameters for M_2 .

Layer	Parameter	#Mult
Conv2d	$CH_{in} = 1, CH_{out} = 4, K = 5, S = 1$	1
Square	-	1
AvgPool2d	kernel size = 2	0
Conv2d	$CH_{in} = 4, CH_{out} = 12, K = 5, S = 1$	1
Square	-	1
AvgPool2d	kernel size = 2	0
Flatten	-	2
FC1	$DAT_{in} = 192, DAT_{out} = 10$	1
Total #Mult	-	7

TABLE 6. Comparison of the average (with standard deviation) inference time in seconds on the MNIST dataset between PyCrCNN and UniHENN for M_2 model architecture.

Model	PyCrCNN	UniHENN
Drop Level	-	0.066 (0.003)
Conv2d	169.182 (0.891)	3.715 (0.048)
Square	88.759 (0.605)	0.140 (0.003)
AvgPool2d	1.955 (0.080)	0.535 (0.012)
Conv2d	520.452 (1.609)	21.697 (0.150)
Square	13.297 (0.116)	0.260 (0.005)
AvgPool2d	0.419 (0.019)	0.875 (0.011)
Flatten	0.000 (0.000)	2.279 (0.020)
FC1	6.527 (0.070)	0.522 (0.014)
Total	800.591 (2.279)	30.089 (0.155)

In Table 6, UniHENN takes 30.089 seconds, making it approximately 26.6 times faster than PyCrCNN, which takes 800.591 seconds. In this experiment, most of the computational time for both UniHENN and PyCrCNN is consumed in the second convolutional layer. These findings highlight the importance of optimizing convolutional layer operations for time-efficient CNN inference in the context of HE.

Furthermore, both Experiments 1 and 2 employ the same input ciphertext, showcasing that UniHENN enables diverse CNN models without requiring re-encryption, provided the supported HE parameters across the models are identical.

C. EXPERIMENT 3: ADAPTIBILITY OF UNIHENN FOR CNN MODELS WITH APPROXIMATE RELU ACTIVATION

In this experiment, we implement a CNN model, denoted as M_3 , with approximate ReLU for the MNIST dataset. Originally, we planned to modify M_2 by replacing the square activation function with approximate ReLU, but the performance was not satisfactory. Therefore, we redesigned the model to achieve better performance.

The model accuracy of M_3 is 98.22%, which is similar to M_2 's accuracy of 98.62%. Table 7 provides detailed specifications for this model architecture.

Table 8 shows a layer-by-layer breakdown of the inference time for the M_3 model architecture. The Flatten and FC1 layers are the most time-consuming, taking an average of 9.603 seconds and 15.557 seconds, respectively. These two layers alone contribute significantly to the total inference time of 29.105 seconds. While UniHENN is significantly

TABLE 7. Detailed parameters for M_3 .

Layer	Parameter	#Mult
Conv2d	$CH_{in} = 1, CH_{out} = 6, K = 3, S = 1$	1
Approx ReLU	$f(x) = 0.375373 + 0.5x + 0.117071x^2$	2
AvgPool2d	kernel size = 2	0
Flatten	-	2
FC1	$DAT_{in} = 1014, DAT_{out} = 120$	1
Approx ReLU	$f(x) = 0.375373 + 0.5x + 0.117071x^2$	2
FC2	$DAT_{in} = 120, DAT_{out} = 10$	1
Total #Mult	-	9

TABLE 8. Average (with standard deviation) inference time in seconds on the MNIST dataset for the M_3 model architecture.

Model	UniHENN
Drop Level	0.035 (0.001)
Conv2d	1.863 (0.029)
Approx ReLU	0.513 (0.011)
AvgPool2d	1.030 (0.024)
Flatten	9.603 (0.081)
FC1	15.557 (0.111)
Approx ReLU	0.045 (0.002)
FC2	0.459 (0.010)
Total	29.105 (0.171)

slower than non-HE models, it provides enhanced privacy and security, which may be crucial for specific applications or compliance requirements. The inference time of 29.105 seconds is still under 30 seconds, suggesting that UniHENN is practical for real-world applications, especially in contexts where data security is paramount. This inference time could be considered acceptable depending on the specific use case and the sensitivity of the data being processed. These results further validate the adaptability and efficiency of UniHENN in handling various CNN architectures with customized functionalities like approximate ReLU.

D. EXPERIMENT 4: ADAPTIBILITY OF UNIHENN FOR LENET-5

In this experiment, we implement the LeNet-5 model [30], denoted as M_4 , with the only modification being the activation function, which is changed from tanh to square. The model accuracy of M_4 is 98.91%. Table 9 provides detailed specifications for this model architecture.

Table 10 shows a layer-by-layer breakdown of the inference time for the M_4 model architecture. The results show that the inference time for M_4 is 740.128 seconds. This is substantially slower than the previous experiments, which could be attributed to the model's increased complexity. The slow performance emphasizes the need for further optimization, especially if UniHENN is to be broadly applied to more complex CNN architectures like LeNet-5 for real-world applications. Note that the convolutional layers are the most time-consuming, highlighting the critical need for optimizing these operations when implementing CNNs using HE. With such a long inference time, the immediate practicality of using this model for real-time or near-real-time applications is limited. However, this could be acceptable

TABLE 9. Detailed parameters for M_4 .

Layer	Parameter	#Mult
Conv2d	$CH_{in} = 1, CH_{out} = 6, K = 5, S = 1$	1
Square	-	1
AvgPool2d	kernel size = 2	0
Conv2d	$CH_{in} = 6, CH_{out} = 16, K = 5, S = 1$	1
Square	-	1
AvgPool2d	kernel size = 2	0
Conv2d	$CH_{in} = 16, CH_{out} = 120, K = 5, S = 1$	1
Square	-	1
Flatten	-	0
FC1	$DAT_{in} = 120, DAT_{out} = 84$	1
Square	-	1
FC2	$DAT_{in} = 84, DAT_{out} = 10$	1
Total #Mult	-	9

TABLE 10. Average (with standard deviation) inference time in seconds on the MNIST dataset for the M_4 model architecture.

Layer	UniHENN
Drop Level	0.035 (0.002)
Conv2d	5.244 (0.048)
Square	0.312 (0.005)
AvgPool2d	0.865 (0.017)
Conv2d	48.222 (0.266)
Square	0.567 (0.010)
AvgPool2d	1.453 (0.020)
Conv2d	668.688 (1.241)
Square	2.613 (0.030)
Flatten	3.995 (0.044)
FC1	7.669 (0.074)
Square	0.011 (0.000)
FC2	0.454 (0.009)
Total	740.128 (1.381)

for services that require strong privacy controls and where data security is a higher priority than speed. For example, in healthcare or financial services, where data may be extremely sensitive, this level of privacy may justify the slower inference times.

E. EXPERIMENT 5: ADAPTABILITY OF UNIHENN FOR CNN MODELS ON COLOR IMAGES

In this experiment, we evaluate the adaptability of UniHENN using the CIFAR-10 color image dataset. We implement a CNN model, denoted as M_5 , which is a modified version of M_4 , to achieve satisfactory accuracy on CIFAR-10. The model accuracy of M_5 is 73.26%. Table 11 provides detailed specifications for this model architecture.

Note that a comparison with TenSEAL is not feasible, as TenSEAL does not process multiple channels, which is essential to process color images. Additionally, we attempted an experiment with PyCrCNN under the same settings but failed to obtain results despite running the experiment for approximately 15 hours. This failure is attributed to PyCrCNN's approach of encrypting each parameter with an individual ciphertext, which demands substantial memory and computational time.

Table 12 presents the experimental results, showing that the total inference time for UniHENN is approximately

TABLE 11. Detailed parameters for M_5 .

Layer	Parameter	#Mult
Conv2d	$CH_{in} = 3, CH_{out} = 16, K = 3, S = 1$	1
Square	-	1
AvgPool2d	kernel size = 2	0
Conv2d	$CH_{in} = 16, CH_{out} = 64, K = 4, S = 1$	1
Square	-	1
AvgPool2d	kernel size = 2	0
Conv2d	$CH_{in} = 64, CH_{out} = 128, K = 3, S = 1$	1
Square	-	1
AvgPool2d	kernel size = 4	0
Flatten	-	1
FC1	$DAT_{in} = 128, DAT_{out} = 10$	1
Total #Mult	-	8

TABLE 12. Average (with standard deviation) inference time in seconds for the M_5 model architecture.

Layer	UniHENN
Drop Level	0.155 (0.004)
Conv2d	8.935 (0.062)
Square	0.703 (0.011)
AvgPool2d	0.429 (0.004)
Conv2d	256.000 (0.867)
Square	1.839 (0.023)
AvgPool2d	1.891 (0.022)
Conv2d	938.446 (1.884)
Square	2.064 (0.019)
AvgPool2d	4.466 (0.049)
Flatten	33.370 (0.162)
FC1	3.146 (0.025)
Total	1251.444 (2.532)

21 minutes. While this may seem long, it is important to note that the convolutional layers are particularly time-consuming, requiring about 256.000 seconds for the first layer and 938.446 seconds for the second. This is consistent with the results of previous experiments. Despite the long inference time, we consider it tolerable given the inherent complexities of performing inference on color images, a capability not offered by alternative solutions.

F. EXPERIMENT 6: EVALUATION OF INFERENCE TIME OF UNIHENN ON GRAYSCALE IMAGES

To assess the adaptability of UniHENN to diverse datasets, we performed experiments using the USPS dataset and the M_6 model architecture. For performance comparison, we also implemented the same model using PyCrCNN. The model accuracy of M_6 is 94.21%. Table 13 provides detailed specifications for this model architecture.

Table 14 shows that UniHENN is approximately 6 times faster than PyCrCNN, with a total inference time of 12.309 seconds compared to PyCrCNN's 71.139 seconds. Interestingly, the FC1 layer in PyCrCNN consumes a significant portion of the time (42.792 seconds), likely due to its large input size of 294. In contrast, although UniHENN also incurs a relatively high computational cost at the FC1 layer, it is substantially less than that of PyCrCNN. This

TABLE 13. Detailed parameters for M_6 .

Layer	Parameter	#Mult
Conv2d	$CH_{in} = 1, CH_{out} = 6, K = 4, S = 2$	1
Square	-	1
Flatten	-	2
FC1	$DAT_{in} = 294, DAT_{out} = 64$	1
Square	-	1
FC2	$DAT_{in} = 64, DAT_{out} = 10$	1
Total #Mult	-	7

TABLE 14. Comparison of the average (with standard deviation) inference time in seconds on the USPS dataset between PyCrCNN and UniHENN for the M_6 model architecture.

Layer	PyCrCNN	UniHENN
Drop Level	-	0.066 (0.002)
Conv2d	14.441 (0.125)	2.662 (0.029)
Square	11.238 (0.090)	0.212 (0.004)
Flatten	0.000 (0.000)	3.026 (0.032)
FC1	42.792 (0.121)	5.948 (0.074)
Square	1.502 (0.026)	0.011 (0.000)
FC2	1.166 (0.015)	0.384 (0.009)
Total	71.139 (0.198)	12.309 (0.092)

TABLE 15. Detailed parameters for M_7 .

Layer	Parameter	#Mult
Conv1d	$CH_{in} = 1, CH_{out} = 2, K = 2, S = 2$	1
Square	-	1
Conv1d	$CH_{in} = 2, CH_{out} = 4, K = 2, S = 2$	1
Flatten	-	1
FC1	$DAT_{in} = 128, DAT_{out} = 32$	1
Square	-	1
FC2	$DAT_{in} = 32, DAT_{out} = 5$	1
Total #Mult	-	7

suggests that UniHENN can efficiently handle FC layers with large input sizes.

G. EXPERIMENT 7: EVALUATION OF INFERENCE TIME OF UNIHENN FOR A 1D CNN MODEL

To evaluate the adaptability of UniHENN for 1D CNN models, we implemented a 1D CNN model, denoted as M_7 , for processing ECG data. This model, a modified version of the 1D CNN model by Abuadbba et al. [4], achieves an accuracy of 96.87%, which is comparable to the 98.90% achieved by Abuadbba et al.'s original model. For performance comparison, the same model was also implemented using PyCrCNN. Table 15 provides detailed specifications for this model architecture.

Table 16 shows that UniHENN is approximately 3.0 times faster than PyCrCNN, recording a total inference time of 5.119 seconds compared to PyCrCNN's 15.514 seconds. This indicates that UniHENN is also more efficient even for relatively smaller models, such as 1D CNNs, where the computational time for convolutional layers is not as extensive. This efficiency positions UniHENN as an ideal choice for privacy-sensitive applications like disease diagnosis systems, aligning well with the principles of HE.

TABLE 16. Average (with standard deviation) inference time in seconds for the M_7 model architecture on the ECG dataset.

Layer	PyCrCNN	UniHENN
Drop Level	-	0.069 (0.007)
Conv1d	0.556 (0.011)	0.113 (0.003)
Square	4.913 (0.043)	0.075 (0.002)
Conv1d	1.081 (0.041)	0.283 (0.006)
Flatten	0.000 (0.000)	1.622 (0.015)
FC1	8.163 (0.067)	2.765 (0.022)
Square	0.559 (0.011)	0.012 (0.000)
FC2	0.242 (0.006)	0.180 (0.016)
Total	15.514 (0.114)	5.119 (0.039)

H. VALIDATION OF INFERENCE RESULTS ON ENCRYPTED DATA

The CKKS scheme operates on approximate complex arithmetic, which can introduce minor errors after homomorphic operations. Therefore, it is crucial to validate UniHENN's inference results by comparing them with the outcomes of the original plaintext inference.

We selected M_4 and M_5 as representative models for validation, as the errors were very small in the case of other models. We evaluated the M_4 model with 2,000 samples from the MNIST dataset and the M_5 model with 2,000 samples from the CIFAR-10 dataset. The results showed that all outputs produced by UniHENN were equivalent to those of the original models without any significant loss in accuracy. This indicates that UniHENN can perform secure and highly accurate inferences under the parameter configurations presented in Table 2.

I. IMPACT OF HE PARAMETERS

We conducted an additional analysis to investigate how the inference time and encrypted inference error vary depending on the parameters of the CKKS scheme.

The CKKS scheme's operation time and decrypted result accuracy are influenced by several parameters, specifically *#slots*, *scale*, and *depth*. Therefore, the inference time for CNN models is intricately linked to these parameters. A detailed analysis of these parameter settings is essential to optimize inference time while maintaining result accuracy. The *#slots* parameter is determined by the degree N of the polynomial ring, where N is half the ring's value. The *scale* parameter represents the precision of floating-point digits, while *depth* is influenced by both *scale* and the polynomial P .

1) INFLUENCE OF DEPTH ON INFERENCE TIME

The operation time increases as the *depth* increases while using a fixed *scale*. Figure 14 shows that, in the M_1 model, the inference time increases linearly with *depth*. This is attributed to the ciphertext size being dependent on *depth*, thereby increasing the computational workload. To achieve optimal performance, the *depth* should align with the number of required multiplications for the model. However, in our architecture, *depth* is predetermined before model selection.

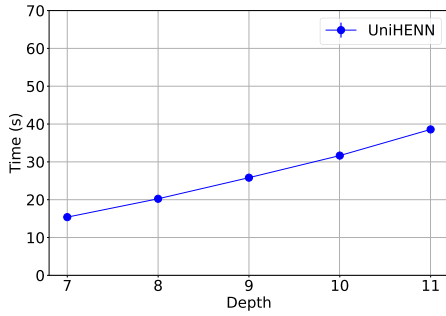


FIGURE 14. Average inference time for M_1 with depth.

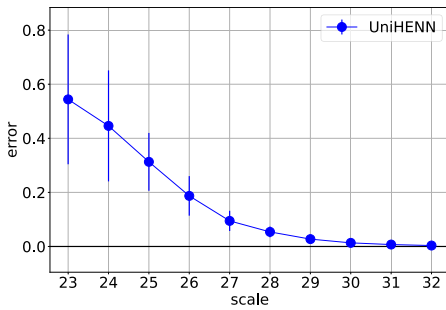


FIGURE 15. Error in M_1 with scale.

To address this, we set the *depth* as high as possible and then optimize by fine-tuning the input ciphertext.

2) IMPACT OF SCALE ON ERROR

We measured the error in the M_1 model's results as a function of *scale*. Detailed findings are presented in Figure 15. Increasing *scale* logarithmically reduces the error. Although a higher *scale* is advantageous, it consequently leads to a lower *depth*, given their inverse relationship due to the fixed $\log Q$ parameter in the CKKS scheme. Therefore, selecting an optimal *scale* is crucial for minimizing error while ensuring sufficient *depth* for CNN inference. A *scale* of 32 ensures 32-bit decimal point precision. From our observations, we note that the error converges toward zero as the *scale* increases. Specifically, using a *scale* value of 30 or higher can significantly reduce the error.

J. COMPARISON OF UNIHENN AND DP-BASED CNN

This section compares our UniHENN framework with differential privacy (DP) for privacy-preserving CNNs, providing insights into the trade-offs between HE and DP-based approaches. We implemented DP-SGD [3] using Opacus [50].

We conducted experiments on the MNIST dataset with model M_4 , comprising 60,000 training and 10,000 test images. The Adam optimizer was used with a learning rate of 0.001, training for 15 epochs with a batch size of 32. For the DP-SGD implementation, we set the maximum per gradient norm to 1.0 and used $\delta = 0.00001$, satisfying the condition $\delta < 1/60,000$ for the MNIST training set size. We varied ϵ

TABLE 17. Accuracy comparison between UniHENN (U) and DP-based CNN (D) for various privacy budgets (ϵ) (30 runs, standard deviations in parentheses).

Framework	U	D ($\epsilon=5$)	D ($\epsilon=1$)	D ($\epsilon=0.5$)	D ($\epsilon=0.1$)
Accuracy (%)	98.91	84.19 (2.59)	84.57 (2.29)	83.73 (1.98)	80.63 (1.54)

TABLE 18. Memory consumption comparison during inference (30 runs, standard deviations in parentheses).

Framework	U	D
Memory (MiB)	3677.64 (1.05)	3.91 (0.10)

from 5 to 0.1 during the 15-epoch training. While there is no definitive guideline for choosing ϵ , NIST's post [39] suggests that ϵ between 0 and 5 provides strong privacy protection. Thus, our selected ϵ values with $\delta = 0.00001$ would represent suitable candidates for DP-SGD-based training.

Tables 17 and 18 present the accuracy and memory consumption results, respectively.

Our results reveal significant trade-offs between HE-based and DP-based privacy-preserving techniques in CNNs. UniHENN achieves superior accuracy (98.91%) compared to the DP-based CNN, which ranges from 84.57% to 80.63% as ϵ decreases. This demonstrates UniHENN's ability to maintain model performance across varying privacy parameters. However, this high accuracy comes at a substantial computational cost: UniHENN consumes about 941 times more memory (3677.64 MiB) than the DP-based approach (3.91 MiB) due to the intensive nature of HE operations.

These findings highlight the critical resource trade-offs in privacy-preserving machine learning, emphasizing the need to balance accuracy and computational efficiency when selecting privacy-preserving techniques for specific applications. The choice between HE and DP approaches will depend on the particular use case, with UniHENN offering superior accuracy at the cost of higher computational resources, while DP-based methods provide a more memory-efficient solution with a trade-off in accuracy.

VIII. LIMITATIONS

We propose an optimized CNN model inference mechanism based on HE, UniHENN. While UniHENN uses batching to reduce inference time, it has three key limitations.

Firstly, complex deep learning models often require a large number of operations, particularly multiplications. In HE, multiplication operations must be limited based on parameter values, or bootstrapping must be used. Due to the extensive multiplication operations in deep learning models, bootstrapping is necessary, significantly slowing computation speed.

Secondly, UniHENN does not currently support multi-core architectures or GPUs. Our deployment uses SEAL-Python, an open-source HE framework suited for CPU and single-thread architectures. Most open-source HE frameworks, including SEAL-Python, are designed this way. Recent

studies have optimized HE operations for GPUs [6], [40], focusing on parallelism to enhance performance. To address these limitations, we plan to extend our architecture to support multi-core and GPU environments. Our future work will involve optimizing key HE operations for parallel execution on GPUs, implementing efficient memory management techniques, and ensuring proper synchronization mechanisms to maximize computational performance.

Finally, UniHENN is designed to reduce computational time through batching, making it efficient for analyzing sensitive data at scale or when inferring multiple data points simultaneously. However, due to its focus on batch operations, UniHENN may be inefficient for inferring single data points. To overcome this, we plan to improve computational efficiency by distributing operations within the convolutional and FC layers in the space where batch operations are performed, thereby increasing overall efficiency.

IX. RELATED WORK

A. LIBRARIES OF HE

Several prominent HE libraries are available, including SEAL [44], Lattigo [38], HELib [1], and OpenFHE [5]. Each offers unique features: SEAL [44] supports BFV and CKKS schemes with optimizations like ciphertext packing. We used SEAL-Python, a Python port of the C++ implementation, for our work. Lattigo [38] implements Ring-LWE-based HE primitives and Multiparty-HE protocols in Go, enabling cross-platform builds and optimized secure computation. HELib [1] supports BGV and CKKS schemes with bootstrapping, focusing on Smart-Vercauteren ciphertext packing and Gentry-Halevi-Smart optimization. It also provides an HE assembly language for low-level control. OpenFHE [5] offers bootstrapping and hardware acceleration using a standard Hardware Abstraction Layer (HAL).

B. MACHINE LEARNING IMPLEMENTATION WITH HE

To provide privacy-preserving ML services, many previous studies implement HE-friendly neural networks. They focus on two challenges: “How to build HE-friendly model architecture, especially, the activation layer” and “How to build an efficient HE-based CNN (or DNN) system.”

In the first case, there were efforts to replace activation functions with polynomial functions. CryptoNets [17] use a square function to replace the ReLU activation function. Chou et al. [11] proposed some techniques to replace activation functions such as ReLU, Softplus, and Swish with low-degree polynomials and use pruning and quantization for the efficiency of homomorphic operations. Ishiyama et al. [20] adopt Google’s Swish activation function with two- and four-degree polynomials with batch normalization to reduce the errors between Swish and approximated polynomials. However, replacing activation functions with low-degree polynomials reduces model accuracy and is only applicable to shallow models. To overcome these limitations, Park et al. [41] proposed the HerPN block, which can replace the batch normalization and ReLU block by utilizing the

Hermite polynomial. Recently, Lee and Lee [31] proposed PP-DNN, a low-latency model optimization solution focusing on convolution and approximate ReLU operations without bootstrapping, supporting CNN models like ResNet-34 with HE. In contrast, UniHENN offers comprehensive optimizations across all layer types, including convolutional, pooling, fully connected, and flatten layers, resulting in more efficient operations overall. Additionally, UniHENN introduces a batch system for multi-input inference and optimizes element sizes for batch operations, which are not addressed in PP-DNN.

In the second case, many studies [16], [21], [41] interact with clients to compute non-linear operations. The server sends the encrypted intermediate results to the client, and the client computes the non-linear functions with decrypted intermediate results. In this way, the model accuracy can be preserved, but the communication overhead is increased.

X. CONCLUSION

This paper presents UniHENN, a privacy-preserving machine learning framework utilizing HE without the `im2col` operation, enhancing compatibility with diverse ML models. We evaluated UniHENN on four public datasets using six 2D CNNs and one 1D CNN, demonstrating accuracy comparable to unencrypted models. UniHENN’s batch processing technique significantly improves efficiency, outperforming the state-of-the-art TenSEAL library by 3.9 times when processing 10 MNIST images using a simple CNN model. This framework opens up new possibilities for privacy-preserving machine learning in real-world applications.

REFERENCES

- [1] (2022). *Helib*. [Online]. Available: <https://github.com/homenc/HELlib>
- [2] (2023). *SEAL-Python*. [Online]. Available: <https://github.com/Huelse/SEAL-Python>
- [3] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, “Deep learning with differential privacy,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2016, pp. 308–318.
- [4] S. Abuadba, K. Kim, M. Kim, C. Thapa, S. A. Camtepe, Y. Gao, H. Kim, and S. Nepal, “Can we use split learning on 1D CNN models for privacy preserving training?” in *Proc. 15th ACM Asia Conf. Comput. Commun. Secur.*, Oct. 2020, pp. 305–318.
- [5] A. Al Badawi et al., “OpenFHE: Open-source fully homomorphic encryption library,” in *Proc. 10th Workshop Encrypted Comput. Appl. Homomorphic Cryptography*, Nov. 2022, pp. 53–63.
- [6] A. Al Badawi, B. Veeravalli, J. Lin, N. Xiao, M. Kazuaki, and A. K. M. Mi, “Multi-GPU design and performance evaluation of homomorphic encryption on GPU clusters,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 2, pp. 379–391, Feb. 2021.
- [7] A. Benaissa, B. Retiat, B. Cebere, and A. E. Belfedhal, “TenSEAL: A library for encrypted tensor operations using homomorphic encryption,” 2021, *arXiv:2104.03152*.
- [8] Z. Brakerski and V. Vaikuntanathan, “Lattice-based FHE as secure as PKE,” in *Proc. 5th Conf. Innov. Theor. Comput. Sci.*, Jan. 2014, pp. 1–12.
- [9] J. H. Cheon, A. Kim, M. Kim, and Y. Song, “Homomorphic encryption for arithmetic of approximate numbers,” in *Advances in Cryptology—ASIACRYPT*. Cham, Switzerland: Springer, 2017, pp. 409–437.
- [10] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachene, “Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds,” in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Security*, 2016, pp. 3–33.
- [11] E. Chou, J. Beal, D. Levy, S. Yeung, A. Haque, and L. Fei-Fei, “Faster CryptoNets: Leveraging sparsity for real-world encrypted inference,” 2018, *arXiv:1811.09953*.

- [12] R. Dathathri, O. Saarikivi, H. Chen, K. Laine, K. Lauter, S. Maleki, M. Musuvathi, and T. Mytkowicz, "CHET: An optimizing compiler for fully-homomorphic neural-network inferencing," in *Proc. 40th ACM SIGPLAN Conf. Program. Lang. Design Implement.*, Jun. 2019, pp. 142–156.
- [13] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2009, pp. 248–255.
- [14] L. Deng, "The MNIST database of handwritten digit images for machine learning research," *IEEE Signal Process. Mag.*, vol. 29, no. 6, pp. 141–142, Nov. 2012.
- [15] S. Disabato, A. Falcetta, A. Mongelluzzo, and M. Roveri, "A privacy-preserving distributed architecture for deep-learning-as-a-service," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2020, pp. 1–8.
- [16] Z. Ghodsi, A. K. Veldanda, B. Reagen, and S. Garg, "CryptoNAS: Private inference on a ReLU budget," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 16961–16971.
- [17] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 201–210.
- [18] S. Halevi and V. Shoup, "Algorithms in HElib," in *Proc. Int. Cryptol. Conf. (CRYPTO)*. Cham, Switzerland: Springer, 2014, pp. 554–571.
- [19] J. J. Hull, "A database for handwritten text recognition research," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 16, no. 5, pp. 550–554, May 1994.
- [20] T. Ishiyama, T. Suzuki, and H. Yamana, "Highly accurate CNN inference using approximate activation functions over homomorphic encryption," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2020, pp. 3989–3995.
- [21] C. Juvekar, V. Vaikuntanathan, and A. Chandrakan, "GAZELLE: A low latency framework for secure neural network inference," in *Proc. 27th USENIX Secur. Symp. (USENIX Security)*, 2018, pp. 1651–1669.
- [22] N. Khalid, A. Qayyum, M. Bilal, A. Al-Fuqaha, and J. Qadir, "Privacy-preserving artificial intelligence in healthcare: Techniques and applications," *Comput. Biol. Med.*, vol. 158, May 2023, Art. no. 106848.
- [23] A. Kim, Y. Song, M. Kim, K. Lee, and J. H. Cheon, "Logistic regression model training based on the approximate homomorphic encryption," *BMC Med. Genomics*, vol. 11, no. S4, pp. 23–31, Oct. 2018.
- [24] T. Kim and H. Kim, "POSTER: Can we use biometric authentication on cloud? Fingerprint authentication using homomorphic encryption," in *Proc. Asia Conf. Comput. Commun. Secur.*, May 2018, pp. 813–815.
- [25] T. Kim, Y. Oh, and H. Kim, "Efficient privacy-preserving fingerprint-based authentication system using fully homomorphic encryption," *Secur. Commun. Netw.*, vol. 2020, pp. 1–11, Feb. 2020.
- [26] S. Kiranyaz, T. Ince, R. Hamila, and M. Gabbouj, "Convolutional neural networks for patient-specific ECG classification," in *Proc. 37th Annu. Int. Conf. IEEE Eng. Med. Biol. Soc. (EMBC)*, Aug. 2015, pp. 2608–2611.
- [27] A. Krizhevsky, V. Nair, and G. Hinton, "CIFAR-10 (Canadian Institute for Advanced Research)," Can. Inst. Adv. Res., 2010, vol. 5, no. 4, p. 1. [Online]. Available: <http://www.cs.toronto.edu/kriz/cifar.html>
- [28] Y. LeCun, B. Boser, J. Denker, D. Henderson, R. Howard, W. Hubbard, and L. Jackel, "Handwritten digit recognition with a back-propagation network," in *Proc. Adv. Neural Inf. Process. Syst.*, 1989.
- [29] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation applied to handwritten zip code recognition," *Neural Comput.*, vol. 1, no. 4, pp. 541–551, Dec. 1989.
- [30] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [31] H. Lee and Y. Lee, "Optimizations of privacy-preserving DNN for low-latency inference on encrypted data," *IEEE Access*, vol. 11, pp. 104775–104788, 2023.
- [32] J.-W. Lee, H. Kang, Y. Lee, W. Choi, J. Eom, M. Deryabin, E. Lee, J. Lee, D. Yoo, Y.-S. Kim, and J.-S. No, "Privacy-preserving machine learning with fully homomorphic encryption for deep neural network," *IEEE Access*, vol. 10, pp. 30039–30054, 2022.
- [33] J. Lee, E. Lee, J.-W. Lee, Y. Kim, Y.-S. Kim, and J.-S. No, "Precise approximation of convolutional neural networks for homomorphically encrypted data," *IEEE Access*, vol. 11, pp. 62062–62076, 2023.
- [34] B. Liu, M. Ding, S. Shaham, W. Rahayu, F. Farokhi, and Z. Lin, "When machine learning meets privacy: A survey and outlook," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 1–36, Mar. 2022.
- [35] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 11966–11976.
- [36] Y. Masahiro, *Fully Homomorphic Encryption Without Bootstrapping*. Saarbrücken, Germany: LAP LAMBERT Academic, 2011.
- [37] George B. Moody and Roger G. Mark, "The impact of the mit-bih arrhythmia database," *IEEE Eng. Med. Biol. Mag.*, vol. 20, no. 3, pp. 45–50, May 2001.
- [38] C. V. Mouchet, J.-P. Bossuat, J. R. Troncoso-Pastoriza, and J.-P. Hubaux, "Lattigo: A multiparty homomorphic encryption library in go," in *Proc. 8th Workshop Encrypted Comput. Appl. Homomorphic Cryptogr.*, 2020, pp. 64–70.
- [39] NIST. (2022). *Differential Privacy: Future Work & Open Challenges*. [Online]. Available: <https://www.nist.gov/blogs/cybersecurity-insights/differential-privacy-future-work-open-challenges>
- [40] A. S. Özcan, C. Aydoğan, E. R. Türkoglu, and E. Savas, "Homomorphic encryption on GPU," *IEEE Access*, vol. 11, pp. 84168–84186, 2023.
- [41] J. Park, M. J. Kim, W. Jung, and J. H. Ahn, "AESPA: Accuracy preserving low-degree polynomial activation for fast private inference," 2022, *arXiv:2201.06699*.
- [42] NAVER Cloud platform. (2023). *Server*. [Online]. Available: <https://www.ncloud.com/product/compute/server>
- [43] Y. Rahulamathavan, "Privacy-preserving similarity calculation of speaker features using fully homomorphic encryption," 2022, *arXiv:2202.07994*.
- [44] Microsoft Res., Redmond, WA, USA. (Jan. 2023). *Microsoft SEAL (Release 4.1)*. [Online]. Available: <https://github.com/Microsoft/SEAL>
- [45] Z. Tian, C. Luo, J. Qiu, X. Du, and M. Guizani, "A distributed deep learning system for web attack detection on edge devices," *IEEE Trans. Ind. Informat.*, vol. 16, no. 3, pp. 1963–1971, Mar. 2020.
- [46] Y. Wang, M. Jia, N. Gao, L. Von Krannichfeldt, M. Sun, and G. Hug, "Federated clustering for electricity consumption pattern extraction," *IEEE Trans. Smart Grid*, vol. 13, no. 3, pp. 2425–2439, May 2022.
- [47] Z. Xing, S. Zhao, W. Guo, F. Meng, X. Guo, S. Wang, and H. He, "Coal resources under carbon peak: Segmentation of massive laser point clouds for coal mining in underground dusty environments using integrated graph deep learning model," *Energy*, vol. 285, Dec. 2023, Art. no. 128771.
- [48] G. Xu, W. Li, and J. Liu, "A social emotion classification approach using multi-model fusion," *Future Gener. Comput. Syst.*, vol. 102, pp. 347–356, Jan. 2020.
- [49] W. Yang, S. Wang, H. Cui, Z. Tang, and Y. Li, "A review of homomorphic encryption for privacy-preserving biometrics," *Sensors*, vol. 23, no. 7, p. 3566, Mar. 2023.
- [50] A. Yousefpour, I. Shilov, A. Sablayrolles, D. Testuggine, K. Prasad, M. Malek, J. Nguyen, S. Ghosh, A. Bharadwaj, J. Zhao, G. Cormode, and I. Mironov, "Opacus: User-friendly differential privacy library in PyTorch," 2021, *arXiv:2109.12298*.
- [51] J. Zhang, C. Li, J. Ye, and G. Qu, "Privacy threats and protection in machine learning," in *Proc. Great Lakes Symp. VLSI*, Sep. 2020, pp. 531–536.
- [52] L. Zhang, A. Rao, and M. Agrawal, "Adding conditional control to text-to-image diffusion models," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 3836–3847.
- [53] H. Choi, S. S. Woo, and H. Kim, "Blind-Touch: Homomorphic encryption-based distributed neural network inference for privacy-preserving fingerprint authentication," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 20, 2024, pp. 21976–21985.
- [54] H. Choi, S. S. Woo, and H. Kim, "Blind-Match: Efficient homomorphic encryption-based 1:N matching for privacy-preserving biometric identification," in *Proc. ACM Int. Conf. Inf. Knowl. Manag.*, 2024.



HYUNMIN CHOI received the M.S. degree from the Graduate School of Information and Communications, Sungkyunkwan University, Seoul, Republic of Korea, in 2022. He is currently pursuing the Ph.D. degree with the Department of Electrical and Computer Engineering, Sungkyunkwan University. He is also with NAVER Cloud Corporation working on security development. His research interests include privacy-enhancing technology and AI security.



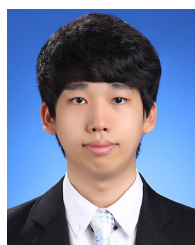
JIHUN KIM is currently pursuing the dual B.S. degree with the Department of Mathematics and the Department of Software, Sungkyunkwan University. His research interests include homomorphic encryption, data privacy, and AI security.



JEONGYONG PARK received the M.S. degree in science (engineering) from the Department of Software, Sungkyunkwan University, in 2023. His current research interests include AI security and data-driven security.



SEUNGHO KIM is currently pursuing the M.S. degree with the Department of Electrical and Computer Engineering, Sungkyunkwan University. His research interests include data privacy and usable security.



WONBIN CHOI received the M.S. degree in information security from the Department of Information Security, School of Cybersecurity, Korea University, Seoul, South Korea, in 2019. He is currently working in security development at NAVER Cloud Corporation. His current research interests include analysis of security vulnerability, formal verification, and information security.



SEONHYE PARK is currently pursuing the Ph.D. degree with the Department of Electrical and Computer Engineering, Sungkyunkwan University. Her research interests include AI security and usable security.



HYOUNGSHICK KIM received the B.S. degree in information engineering from Sungkyunkwan University, in 1999, the M.S. degree in computer science from Korea Advanced Institute of Science and Technology (KAIST), in 2001, and the Ph.D. degree from the Computer Laboratory, University of Cambridge, in 2012. He completed Postdoctoral work at the Department of Electrical and Computer Engineering, The University of British Columbia. He was a Senior Engineer at Samsung Electronics, from 2004 to 2008, and a Distinguished Visiting Researcher at CSIRO, Data61, from 2019 to 2020. He is currently an Associate Professor with the Department of Computer Science and Engineering, Sungkyunkwan University. His research interests include usable security, security vulnerability analysis, and data-driven security.

...